IN THIS CHAPTER

- Working with sets
- Creating and modifying a linked list
- Identifying problems with sets and linked lists

Chapter 2
Sets and Linked Lists

An array can be handy for storing data that consists of the same type of information, such as integers. However, arrays can often be too restrictive. You must define the size of an array ahead of time, and you can only store one data type. In addition, searching, rearranging, or deleting data from an array can be cumbersome.

To solve the problems of arrays, programming languages have come up with a variety of solutions. The obvious solution involves modifying the way arrays work, such as letting you create *dynamic* (resizable) arrays that can store a special `Variant` data type. Unlike an `Integer` data type (which can hold only whole numbers) or a `String` data type (which can hold only text), a `Variant` data type can hold both numbers and text. (Not all programming languages offer dynamic arrays or `Variant` data types.)

Instead of modifying the way arrays work, many programming languages allow you to create other types of data structures. Two popular alternatives to arrays are sets and lists. This chapter shows you how to use sets and lists when arrays are too restrictive and cumbersome.

Copyright © 2022. John Wiley & Sons, Incorporated. All rights reserved.

Wang, Wallace. Beginning Programming All-In-One for Dummies, John Wiley & Sons, Incorporated, 2022. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=6979008.
Created from univ-people-ebooks on 2026-10-12 00:14:42.

Using Sets

If someone showed you the days of the week (Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, and Sunday), you'd know that those days are related as a group that defines a week, but how do you tell a computer that? Here's one way:

1. **Create an array to hold seven separate variables, like this:**

```
Dim Day(6) as String
```

2. **Assign each variable a different name, like this:**

```
Day(0) = "Monday"
Day(1) = "Tuesday"
Day(2) = "Wednesday"
Day(3) = "Thursday"
Day(4) = "Friday"
Day(5) = "Saturday"
Day(6) = "Sunday"
```

This would store all the days of the week inside a single array variable.

However, there's a simpler way to lump related data together: Use a data structure called a *set.* Like an array, a set groups data in a single variable name, but a set has several advantages:

- You don't have to define a fixed size ahead of time.
- You don't have to identify each chunk of data with an index number.

Defining a set *lists* all the data you want to store, as shown in this Python programming language example:

```
days = {'Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday',
   'Saturday', 'Sunday'}
```

In this Python language example, the variable `days` contains the entire set or group of the days. To print the contents of this set, you can use a `print` command followed by the name of the set like this:

```
print (days)
```

Copyright © 2022. John Wiley & Sons, Incorporated. All rights reserved.

Wang, Wallace. Beginning Programming All-In-One for Dummies, John Wiley & Sons, Incorporated, 2022. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=6979008.
Created from univ-people-ebooks on 2026-10-12 00:14:42.

This command may print the following:

```
{'Monday', 'Sunday', 'Thursday', 'Friday', 'Wednesday',
  'Saturday', 'Tuesday'}
```


REMEMBER

Sets don't store data in any particular order. If you print out the contents of a set, the items in that set could come out in a different order than the way you may have stored those items in the set.

Adding and deleting data in a set

To add data to or delete data from a set, use the add and delete commands. In Python, the add command is `add`, and the delete command is `remove`.


REMEMBER

Every programming language uses slightly different names for the same command, so don't worry about the particular command names. You just need to understand the basic principles.

To add more data to a set in Python, you have to identify the set name followed by the `add` command and the data you want to add. So, if you had a set called `clubmembers`, you could use the following commands:

```
clubmembers = {'Bill Evans', 'John Doe', 'Mary Jacobs'}
```

You could add a new name to that set by using the following command:

```
clubmembers.add('Donna Volks')
```

To remove a name from a set, you have to identify the set name, use the `remove` command, and specify which data you want to remove, like this:

```
clubmembers.remove('Bill Evans')
```

Figure 2-1 shows how `add` and `remove` commands change the contents of a set:

- When you delete data from a set, the set is just one item smaller.
- When you delete data from an array, you're left with an empty space in the array.


TIP

To remove all data in a set, most programming languages include a command that clears out an entire set. In Python, the command is `clear`:

```
clubmembers.clear()
```


Copyright © 2022. John Wiley & Sons, Incorporated. All rights reserved.

Wang, Wallace. Beginning Programming All-In-One for Dummies, John Wiley & Sons, Incorporated, 2022. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=6979008.
Created from univ-people-ebooks on 2026-10-12 00:14:42.

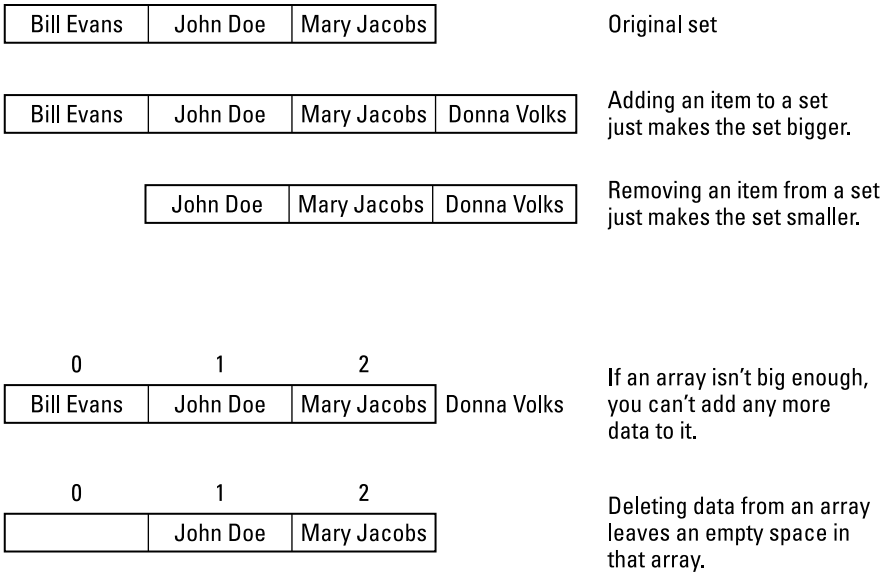


FIGURE 2-1: Adding and removing data in a set is easier than adding and removing data in an array.

Checking for membership

If you store a bunch of names in an array, how could you verify whether a specific name is in that array? Older programming languages often had to examine each element in the array and compare it with the name you're looking for until you either found a match or reached the end of the array.

Sets avoid this problem by making it easy to check whether a chunk of data is stored in a set. If you had a list of country club members stored in a set, it might look like this in Python:

```
clubmembers = {'Bill Evans', 'John Doe', 'Mary Jacobs'}
```

To check whether a name is in a set (that is, a *member* of that set), use a simple `in` command like this:

```
'John Doe' in clubmembers
```

If this command finds the name `John Doe` in the set defined by the `clubmembers` set, this would return a `True` value. If this command can't find the name `John Doe` in the `clubmembers` set, the command would return a `False` value.



Another way to check for membership is to use the `not` command with the `in` command like this:

```
'Hugh Lake' not in clubmembers
```


Copyright © 2022. John Wiley & Sons, Incorporated. All rights reserved.

Wang, Wallace. Beginning Programming All-In-One for Dummies, John Wiley & Sons, Incorporated, 2022. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=6979008.
Created from univ-people-ebooks on 2026-10-12 00:14:42.

This command asks the computer whether the name `Hugh Lake` is not in the `clubmembers` set. In this case, the name `Hugh Lake` is not in the `clubmembers` set, so the preceding command would return a `True` value.

If you used the following command to check whether the name `John Doe` is not in the `clubmembers` set, the following command would return a `False` value because the name `John Doe` is in the `clubmembers` set:

```
'John Doe' not in clubmembers
```

Avoiding duplicate data

One huge advantage of sets is that they won't store duplicate data. With an array, it's possible to store duplicate data multiple times, like this:

```
var nameArray = ["Bob", "Sally", "Bob", "Peter", "Bob",
  "Hannah"]
```

In this array, "Bob" gets stored three times. However, sets automatically exclude duplicate data. That way you can be sure that all data in a set is unique. Let's create a set in Swift and print it, like this:

```
var nameSet: Set = ["Bob", "Sally", "Bob", "Peter", "Bob", "Hannah"]
print(nameSet)
```

The first line creates a set and fills it with strings. Notice that `"Bob"` appears three times. The second line prints the set, but because the set strips out duplicate data, it prints the following:

```
["Bob", "Sally", "Peter", "Hannah"]
```

Manipulating two sets

A set by itself can be handy for grouping related data together, but if you have two or more sets of data, you can manipulate the data in both sets. For example, suppose you have a set of country club members and a second set of people applying for membership.

You can combine both sets together to create a third set (a *union*), find the common data in both sets (an *intersection*), or take away the common data in both sets (the *difference*).

Copyright © 2022. John Wiley & Sons, Incorporated. All rights reserved.

Wang, Wallace. Beginning Programming All-In-One for Dummies, John Wiley & Sons, Incorporated, 2022. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=6979008.
Created from univ-people-ebooks on 2026-10-12 00:14:42.

Combining two sets into a third set with the union command

A union simply takes data from two sets and smashes them together to create a third set that includes all data from the first two sets, as shown in Figure 2-2.

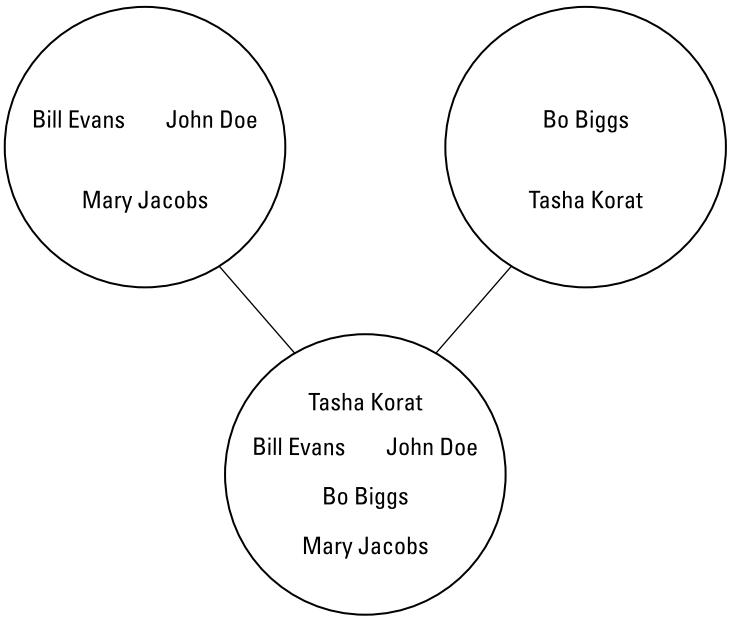


FIGURE 2-2: The `union` command combines data from two sets to create a third set.

To use the `union` command in Python, you need to identify the two set names with the `union` command. Suppose you had one set called `club_members` and another set called `applicants`, as follows:

```
club_members = {'Bill Evans', 'John Doe', 'Mary Jacobs'}
applicants = {'Bo Biggs', 'Tasha Korat'}
```

Now if you wanted to combine the data in both sets and store it in a third set called `every_member`, you could use the `union` command as follows:

```
every_member = club_members.union(applicants)
```

This creates a third set called `every_member` and stores the data from both sets into the `every_member` set. The data in the other sets isn't modified in any way.

Copyright © 2022. John Wiley & Sons, Incorporated. All rights reserved.

Wang, Wallace. Beginning Programming All-In-One for Dummies, John Wiley & Sons, Incorporated, 2022. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=6979008.
Created from univ-people-ebooks on 2026-10-12 00:14:42.

REMEMBER

The order in which you define the two sets is irrelevant:

- You can put the `club_members` set name first, like this:

```
every_member = club_members.union(applicants)
```

- You could switch the two set names around, like this:

```
every_member = applicants.union(club_members)
```

The end result is identical to creating a third set and dumping data from both sets into this third set. If you combine two sets that happen to contain one or more identical chunks of data, the *union* (combination of the two sets) is smart enough not to store duplicate data twice.

Combining the common elements of two sets into a third set with the intersection command

Whereas the `union` command combines two sets into one, the `intersection` command creates a third set that only includes data stored in both sets, as shown in Figure 2-3.

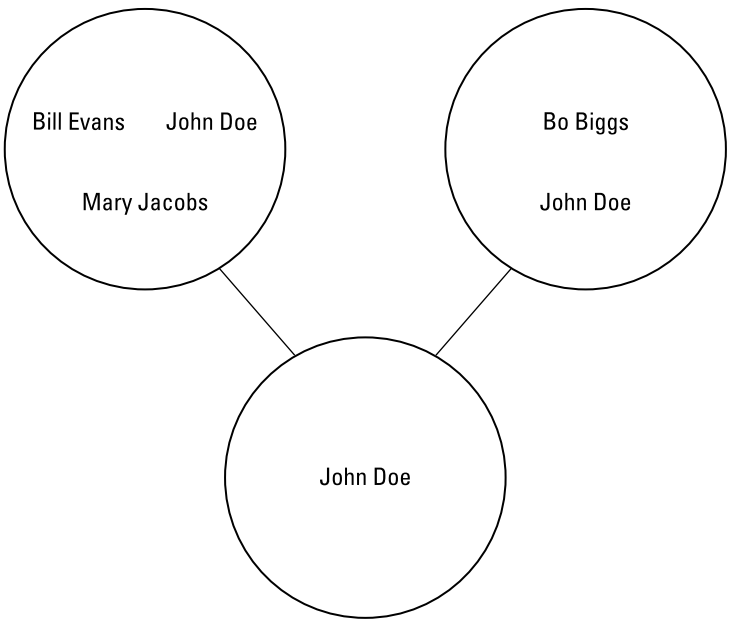


FIGURE 2-3: The `intersection` command takes only data common in both sets and stores that data in a third set.

Copyright © 2022. John Wiley & Sons, Incorporated. All rights reserved.

Wang, Wallace. Beginning Programming All-In-One for Dummies, John Wiley & Sons, Incorporated, 2022. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=6979008.
Created from univ-people-ebooks on 2026-10-12 00:14:42.

To use the `intersection` command in Python, you need to identify the two set names with the `intersection` command. Suppose you had one set called `club_members` and another set called `politicians`, as follows:

```
club_members = {'Bill Evans', 'John Doe', 'Mary Jacobs'}
politicians = {'Bo Biggs', 'John Doe'}
```

Now if you wanted to find only that data stored in both sets, you could use the `intersection` command to store this data in a third set, as follows:

```
new_set = club_members.intersection(politicians)
```

This creates a third set — `new_set` — which contains the name `John Doe`. The other names are omitted because they aren't in *both* original sets.

Combining the different elements of two sets into a third set with the difference command

If you have two sets, you may want to identify all the data stored in one set that isn't stored in a second set. To do this, you'd use the `difference` command, as shown in Figure 2-4.

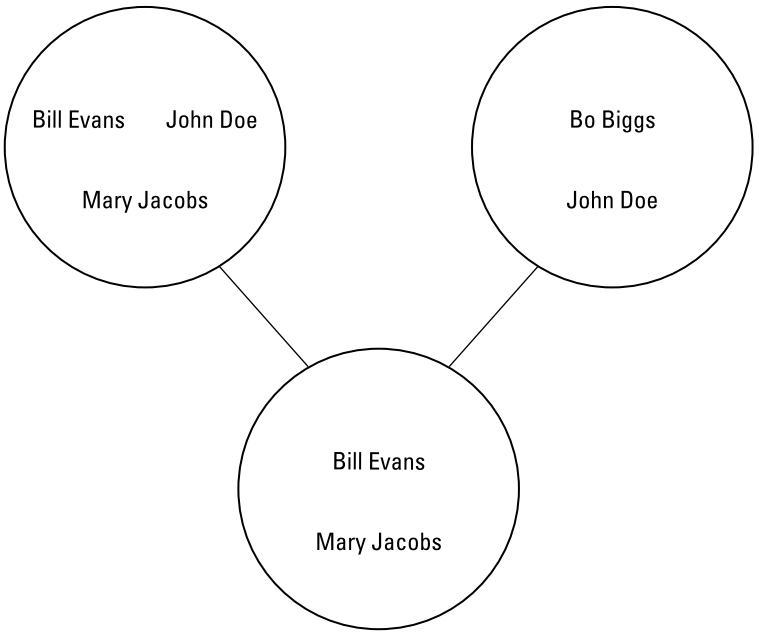


FIGURE 2-4: The `difference` command strips out data in common with a second set.

Copyright © 2022. John Wiley & Sons, Incorporated. All rights reserved.

Wang, Wallace. Beginning Programming All-In-One for Dummies, John Wiley & Sons, Incorporated, 2022. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=6979008.
Created from univ-people-ebooks on 2026-10-12 00:14:42.

To use the `difference` command in Python, you need to identify the two set names with the `difference` command. Suppose you had one set called `club_members` and another set called `politicians`, as follows:

```
club_members = {'Bill Evans', 'John Doe', 'Mary Jacobs'}
politicians = {'Bo Biggs', 'John Doe'}
```

Now if you wanted to store data from the first set that is not in the second set, you could use the `difference` command to store this data in a third set, as follows:

```
newset = club_members.difference(politicians)
```

This creates a third set — `newset` — which contains the names `Bill Evans` and `Mary Jacobs`.



WARNING

The third set does not contain the name `Bo Biggs`. That's because the order in which you list the sets determines how the `difference` command works. If you list the sets in this order:

```
newset = club_members.difference(politicians)
```

You're telling the computer to take all the data from the first set (`club_members`), find all the data common in both the `club_members` and `politicians` sets, and remove that common data from the first set. Now take what's left and dump this data into the `newset` (refer to Figure 2-4).

If you switched the commands around, like this, you'd get an entirely different result:

```
newset = politicians.difference(club_members)
```

This tells the computer to take the data stored in the `politicians` set, find all the data common in both the `politicians` and `club_members` sets, and remove this common data from the `politicians` set. Now store what's left in the `newset`, as shown in Figure 2-5.

Copyright © 2022. John Wiley & Sons, Incorporated. All rights reserved.

Wang, Wallace. Beginning Programming All-In-One for Dummies, John Wiley & Sons, Incorporated, 2022. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=6979008.
Created from univ-people-ebooks on 2026-10-12 00:14:42.

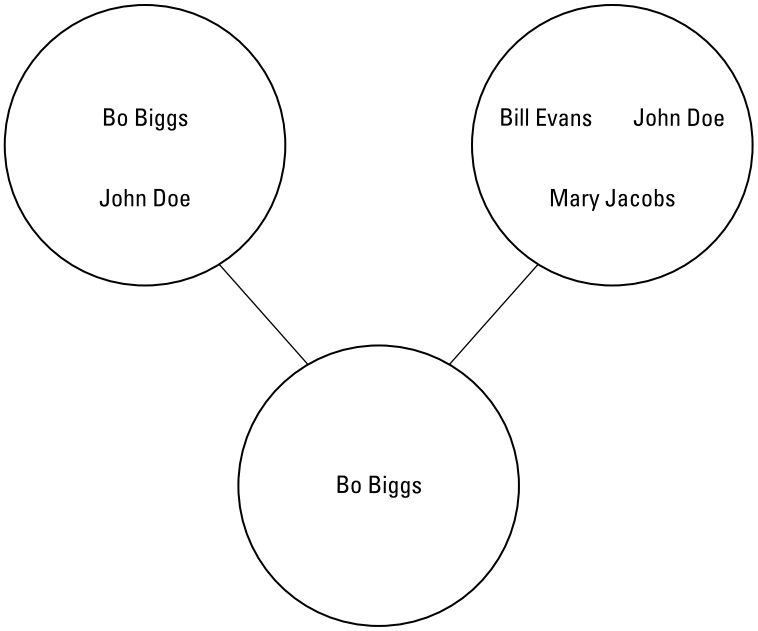


FIGURE 2-5: The order in which you list set names with the `difference` command determines which data gets stored in the third set.

Using Linked Lists

Sets are handy for lumping related data in a group. However, sets aren't organized. So, if you want to group related data together and keep this data sorted, you can use another data structure: a *linked list.*

Whereas an array creates a fixed location for storing data (think of an egg carton), a linked list more closely resembles beads tied together by a string. It's impossible to rearrange an array (just as you can't rip an egg carton apart and put it back together again in a different way). However, you can rearrange a linked list easily just as you can rearrange beads on a string.

The basic element of a linked list is a *node,* which is just a *structure* (see Book 3, Chapter 1) that contains two parts:

- A pointer
- A variable for storing data

Figure 2-6 shows how the parts of a linked list work.

Copyright © 2022. John Wiley & Sons, Incorporated. All rights reserved.

Wang, Wallace. Beginning Programming All-In-One for Dummies, John Wiley & Sons, Incorporated, 2022. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=6979008.
Created from univ-people-ebooks on 2026-10-12 00:14:42.

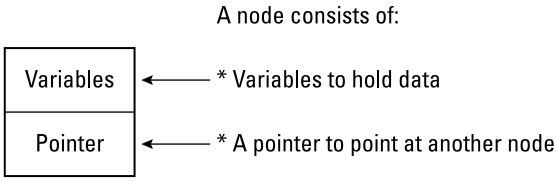


FIGURE 2-6: A node consists of a pointer and one or more variables to store data.



TECHNICAL STUFF

Pointers are often used to access specific locations in the computer's memory. If you've stored data in memory and you need to share that data, you could make duplicate copies of that data, which would take up space. Or you could use a pointer, which allows different parts of a program to access that same data without creating duplicate copies of that data and wasting space.



WARNING

Accessing the computer's memory is like probing your brain with a sharp needle. If you know what you're doing, pointers can give you complete control over a computer, but if you make a mistake, pointers can mess up the computer's memory, causing the entire operating system to crash.

Creating a linked list

A linked list consists of one or more identical nodes that can hold the same number and types of data, such as a `string` and an `integer`. Each time you create a node, you have to define the following:

- The data to store in the node
- The node to point at



REMEMBER

Nodes in a linked list must all contain the same data types, much like an array.

A node can store either

- A single data type (such as a string)
- Another data structure (such as a structure or an array)

Each time you create a node, the node is empty. To make the node useful, you must store data in that node and define which node to point at:



TECHNICAL STUFF

- The first node you create simply points at *nothing.*

  The term `nil` or `null` is commonly used in programming languages to represent the absence of data.

Copyright © 2022. John Wiley & Sons, Incorporated. All rights reserved.

Wang, Wallace. Beginning Programming All-In-One for Dummies, John Wiley & Sons, Incorporated, 2022. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=6979008.
Created from univ-people-ebooks on 2026-10-12 00:14:42.

» Any additional nodes you create point to the previous existing nodes, so you create a daisy-chain effect of nodes linked to one another by pointers, as shown in Figure 2-7.

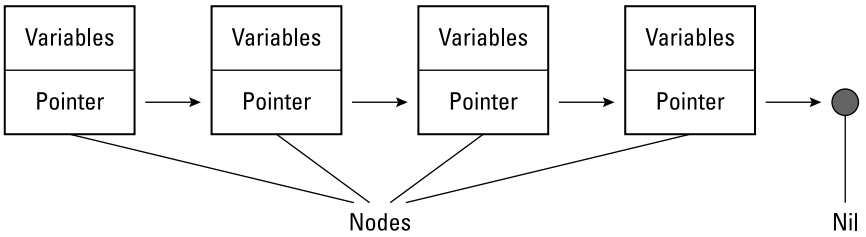


FIGURE 2-7: A linked list stores data in each node that points to another node.

Modifying a linked list

After you create a linked list and store data in it, you can easily modify that linked list by rearranging the pointers, as shown in Figure 2-8.

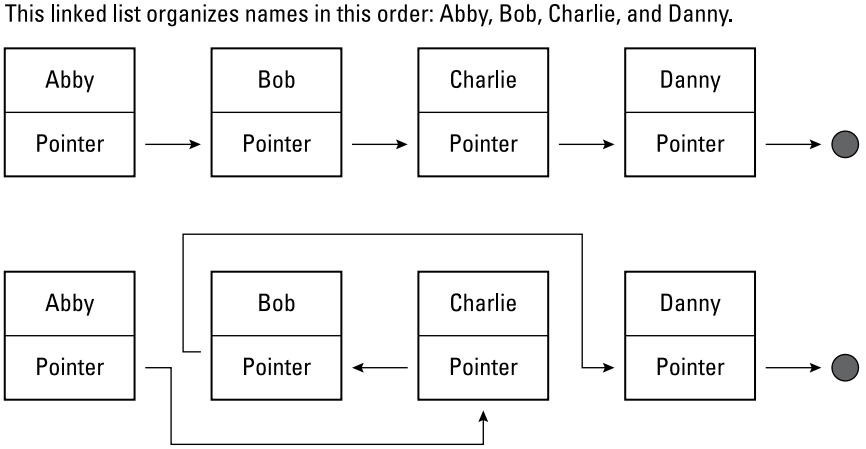


FIGURE 2-8: Rearranging the order of a linked list is as simple as rearranging pointers.

To add data to a linked list, you can just rearrange pointers to include the new data in any position within the linked list, as shown in Figure 2-9.

To delete data from a linked list, you can delete an entire node. Then you must change the pointers to keep your linked list together, as shown in Figure 2-10. Unlike arrays, linked lists give you the flexibility to rearrange data without physically moving and copying it to a new location.

Copyright © 2022. John Wiley & Sons, Incorporated. All rights reserved.

Wang, Wallace. Beginning Programming All-In-One for Dummies, John Wiley & Sons, Incorporated, 2022. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=6979008.
Created from univ-people-ebooks on 2026-10-12 00:14:42.

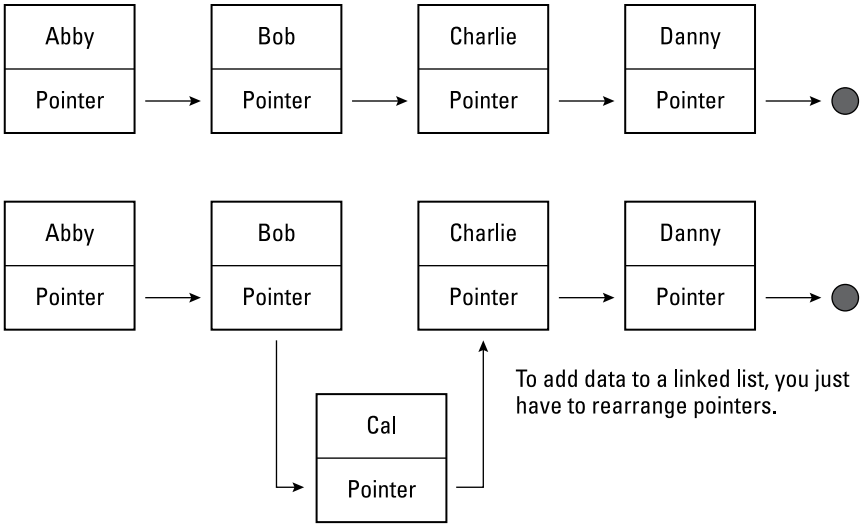


FIGURE 2-9: Adding data from a linked list also involves rearranging pointers.

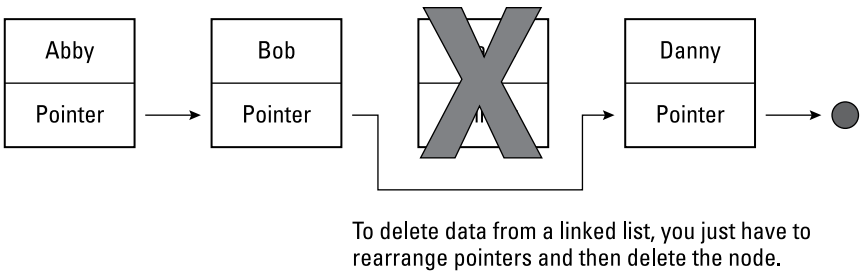


FIGURE 2-10: Deleting data from a linked list involves removing data and then rearranging pointers.

Linked lists also let you add data anywhere just by rearranging the pointers (refer to Figure 2-9 and Figure 2-10). By using linked lists, you can add, delete, and rearrange data quickly and easily.

Creating a double linked list

An ordinary linked list contains pointers that point in one direction only. That means if you start at the beginning of a linked list, you can always browse the data in the rest of the linked list. However, if you start in the middle of a linked list, you can never browse the previous nodes.

To fix this problem, you can also create *a double linked list*, which essentially creates nodes that contain two pointers:

- One pointer points to the *previous* node in the list.
- The other pointer points to the *next* node in the list.

Copyright © 2022. John Wiley & Sons, Incorporated. All rights reserved.

Wang, Wallace. Beginning Programming All-In-One for Dummies, John Wiley & Sons, Incorporated, 2022. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=6979008.
Created from univ-people-ebooks on 2026-10-12 00:14:42.

By using double linked lists, you can easily browse a linked list in both directions, as shown in Figure 2-11.

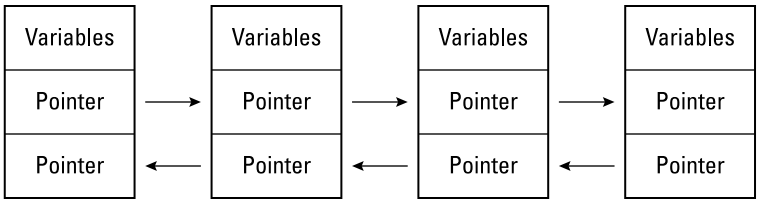


FIGURE 2-11: A double linked list lets you traverse a linked list in both directions.

Another type of linked list you can create is a *circular linked list*, as shown in Figure 2-12. Circular linked lists more closely resemble a doughnut with no beginning or ending node. For more flexibility, you can even create a double linked, circular linked list, which lets you traverse the list both backward and forward.

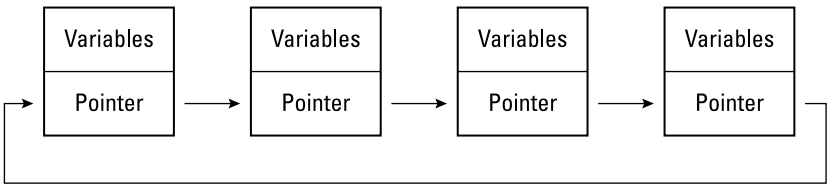


FIGURE 2-12: A circular linked list has no beginning or end.



TECHNICAL STUFF

Linked lists are often used to create other types of data structures, such as *queues*, *graphs*, and *binary trees*, which you can read more about in Chapters 4 and 5 of this minibook.

Drawbacks of Sets and Linked Lists

Sets make it easy to group and manipulate related data, but unlike arrays, there isn't always an easy way to access and retrieve individual items in a set. Sets are best used for treating data as a group rather than as separate chunks of data.

Linked lists are much more flexible than arrays for adding, deleting, and rearranging data. However, the two biggest drawbacks of linked lists are the complexity needed to create them and the potentially dangerous use of pointers.

Copyright © 2022. John Wiley & Sons, Incorporated. All rights reserved.

Wang, Wallace. Beginning Programming All-In-One for Dummies, John Wiley & Sons, Incorporated, 2022. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=6979008.
Created from univ-people-ebooks on 2026-10-12 00:14:42.

Problems with pointers

The most common problem with linked lists occurs when pointers fail to point to either `nil` or a valid node of a linked list. If you delete a node from a linked list but forget to rearrange the pointers, you essentially cut your linked list in two, as shown in Figure 2-13.

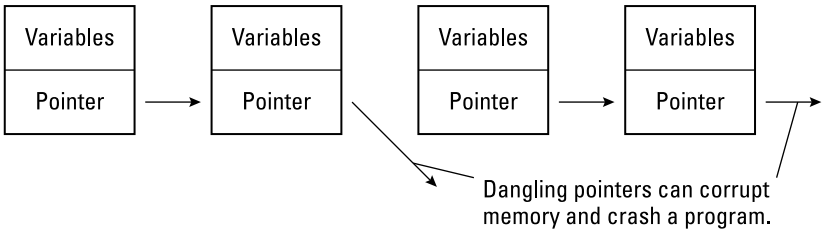


FIGURE 2-13: Pointers must always point to a valid node of a linked list.

Even worse, you could create a *dangling pointer* (the pointer no longer points to a valid node). Dangling pointers can wind up pointing to any part of the computer's memory, usually with catastrophic consequences that crash the entire computer.



TECHNICAL STUFF

Dangling pointers are a common cause of problems using languages like C or C++. Other languages (such as C#, Java, Python, and Swift) prevent dangling pointers.

Problems with accessing data

Accessing data in an array is easy. You can access data by its index number or by starting at the beginning of the array and browsing through each element until you reach the end of the array.

If you want to access data stored in a linked list, you have to start at the beginning. If you start in the middle, you can never go backward to the front of the linked list (unless you're using a double linked list). Arrays let you jump straight to specific data by using an index number. Linked lists don't offer that same feature.



REMEMBER

For ease in *storing, adding,* and *removing* data, linked lists are more flexible than arrays. For *retrieving* data, arrays are much simpler and faster.

Copyright © 2022. John Wiley & Sons, Incorporated. All rights reserved.

Wang, Wallace. Beginning Programming All-In-One for Dummies, John Wiley & Sons, Incorporated, 2022. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=6979008.
Created from univ-people-ebooks on 2026-10-12 00:14:42.

COMPLEXITY OF CREATING LINKED LISTS AND POINTERS

Creating and managing a linked list with all its pointers is easy in theory, but writing the code to create and manage a linked list can get complicated in a hurry. The more confusing and complicated the code, the more likely errors will creep in and cause your linked list to not work at all or to work in unexpected ways.

To show you how confusing pointers and nodes can be to create, study the following Pascal programming language examples. Pascal is actually designed to be an easy-to-read language, but even creating linked lists in Pascal can get clumsy. (Don't worry too much about the details of the Pascal code. Just skim through the examples and follow along the best you can. If you get confused, you can see how implementing linked lists in any programming language can get messy.)

To create a linked list, you must first create a node, which is a *structure.* (In Pascal, structures are called *records.*) To define a structure in Pascal, you could do this:

```
Type
  NodePtr = ^Node;
  Node = RECORD
    Data : String;
    Next : NodePtr;
  END;
```

This Pascal code creates a `NodePtr` variable, which represents a pointer to the `Node` structure (record). The caret symbol (^) defines a pointer, whereas the `Node` name defines what structure the pointer can point at.

The `Node` structure declares two variables: `Data` and `Next`. The `Data` variable holds a string (although you can change this to `Integer` or any other data type). The `Next` variable represents a pointer to the `Node` record. Every node absolutely must have a pointer because pointers are how the nodes can point, or *link,* together to form a linked list.

If this were a double linked list, you'd have two variables (such as `Previous` and `Next`) declared as node pointers like this:

```
Type
  NodePtr = ^Node;
  Node = RECORD
    Data : String;
```

Copyright © 2022. John Wiley & Sons, Incorporated. All rights reserved.

Wang, Wallace. Beginning Programming All-In-One for Dummies, John Wiley & Sons, Incorporated, 2022. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=6979008.
Created from univ-people-ebooks on 2026-10-12 00:14:42.

```
    Previous, Next : NodePtr;
  END;
```

After you define a node as a structure, you can't use that node until you declare a variable to represent that node, like this:

```
Var
  MyNode : NodePtr;
```

After you declare a variable to represent a pointer to a node (structure), you must create a new node, stuff data into it, and then set its pointer to point at something, such as `NIL` or another node:

```
Begin
  New (MyNode);        (* Creates a new node *)
  With MyNode^ do      (* Stores data in the node *)
    Begin
      Data := 'Joe Hall';
      Next := NIL;
    End;
End.
```

To create a linked list in a language like Pascal, you must

1. **Define a node structure.**
2. **Declare a pointer to that node (structure).**
3. **Declare a variable to represent that pointer.**

   Now you can use your node to store data and link with other nodes.

If you mess up on any one of those steps, your linked list won't work, and because linked lists use pointers, your pointers could point anywhere in memory, causing all sorts of random problems. The bottom line: Linked lists are a powerful and flexible data structure, but they come at a price of added complexity for the programmer. Accidentally create one dangling pointer, and you can bring your entire program crashing to a halt.

Copyright © 2022. John Wiley & Sons, Incorporated. All rights reserved.

Wang, Wallace. Beginning Programming All-In-One for Dummies, John Wiley & Sons, Incorporated, 2022. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=6979008.
Created from univ-people-ebooks on 2026-10-12 00:14:42.

Copyright © 2022. John Wiley & Sons, Incorporated. All rights reserved.

Wang, Wallace. Beginning Programming All-In-One for Dummies, John Wiley & Sons, Incorporated, 2022. ProQuest Ebook Central,
http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=6979008.
Created from univ-people-ebooks on 2026-10-12 00:14:42.